
MINI PROJECT – 1

SQL Injection Lab

HackerHub8 Internship – Cyber Security

//Screenshots are added at the last.

Project Title

SQL Injection Vulnerability Demonstration using PHP and MySQL

Objective

The objective of this project is to understand how SQL Injection vulnerabilities occur in web applications when user input is not properly handled.

This project demonstrates an intentionally vulnerable search application built using **PHP** and **MySQL** in a local sandbox environment. It helps learners understand why directly using user input in SQL queries is insecure and why secure coding practices are important.

Problem Statement

Many web applications accept user input through forms such as login pages, search boxes, and registration forms.

If the application directly inserts this input into SQL queries without proper validation or parameterization, the application becomes vulnerable to SQL Injection.

The purpose of this project is to demonstrate this insecure coding practice in a controlled learning environment.

Technologies Used

PHP

MySQL

XAMPP

Apache Server

phpMyAdmin

HTML

CSS

Visual Studio Code

Software Requirements

XAMPP

Visual Studio Code

Google Chrome / Microsoft Edge

Windows Operating System (or Linux with Apache, PHP, and MySQL)

Project Folder Structure

SQLInjectionLab/

|— db.php

|— index.php

|— search.php

|— style.css

Database Used

Database Name

vulnerable_lab

Table Name

users

Columns

Column	Data Type
--------	-----------

id	INT
----	-----

username	VARCHAR
----------	---------

email	VARCHAR
-------	---------

password	VARCHAR
----------	---------

Sample Data

The database contains sample users.

Example:

ID	Username	Email
----	----------	-------

1	Alice	alice@gmail.com
2	Bob	bob@gmail.com
3	Charlie	charlie@gmail.com
4	David	david@gmail.com
5	Emma	emma@gmail.com

Project Workflow

The workflow of the application is as follows:

User

↓

Search Box

↓

PHP Application

↓

SQL Query

↓

MySQL Database

↓

Matching Record Displayed

Working of the Project

Step 1

The user opens the application in the browser.

Example:

<http://localhost/SQLInjectionLab>

Step 2

The application displays a search box.

The user enters a username.

Example

Alice

Step 3

The username entered by the user is sent to the PHP application.

Step 4

PHP receives the user input using the GET method.

Step 5

The application creates an SQL query using the entered username.

The query searches the **users** table for a matching username.

Step 6

The SQL query is executed in the MySQL database.

Step 7

If a matching record exists, the user details are displayed.

Otherwise,

No User Found

is displayed.

Features

PHP and MySQL Integration

User Search Functionality

Database Connectivity

HTML Form

Dynamic Search Results

Simple User Interface

Educational SQL Injection Demonstration

Why is this Project Vulnerable?

The application directly inserts user input into the SQL query without validation or parameterization.

This is intentionally done because the project is designed as a cybersecurity learning lab.

In real-world applications, this coding style is unsafe because it allows user input to influence the SQL query directly.

SQL Injection Concept

SQL Injection is a web application vulnerability that occurs when user input is incorporated directly into SQL statements instead of being safely handled.

In this project, the vulnerability exists because the application accepts input and constructs SQL queries without using secure techniques such as prepared statements.

Error-Based SQL Injection (Concept)

Error-Based SQL Injection is a type of SQL Injection where database error messages reveal information about the application's SQL queries or database structure.

If database errors are displayed to users, they may expose implementation details that can help an attacker understand the application.

This project is intended to illustrate why exposing database errors can be risky.

Blind SQL Injection (Concept)

Blind SQL Injection occurs when database errors are not displayed.

Instead of seeing error messages, an attacker attempts to infer information by observing differences in the application's responses.

This project introduces the concept so learners understand that hiding database errors alone is not sufficient if the underlying vulnerability still exists.

Difference Between Error-Based and Blind SQL Injection

Error-Based SQL Injection	Blind SQL Injection
Database errors are visible.	Database errors are hidden.
Information is obtained from error messages.	Information is inferred from application behavior.
Easier to identify the vulnerability.	More difficult to identify.

Advantages of the Project

Helps understand SQL Injection concepts.

Demonstrates insecure coding practices in a safe environment.

Improves understanding of PHP–MySQL interaction.

Useful for cybersecurity learning and awareness.

Provides practical experience with web application security concepts.

Limitations

Designed only for educational purposes.

Intentionally insecure.

Should never be deployed on a public server.

Not suitable for production environments.

Learning Outcomes

After completing this project, I learned:

How PHP connects to MySQL.

How SQL queries are executed.

How user input flows through a web application.

Why SQL Injection vulnerabilities occur.

The conceptual difference between Error-Based SQL Injection and Blind SQL Injection.

The importance of secure coding practices for database-driven applications.

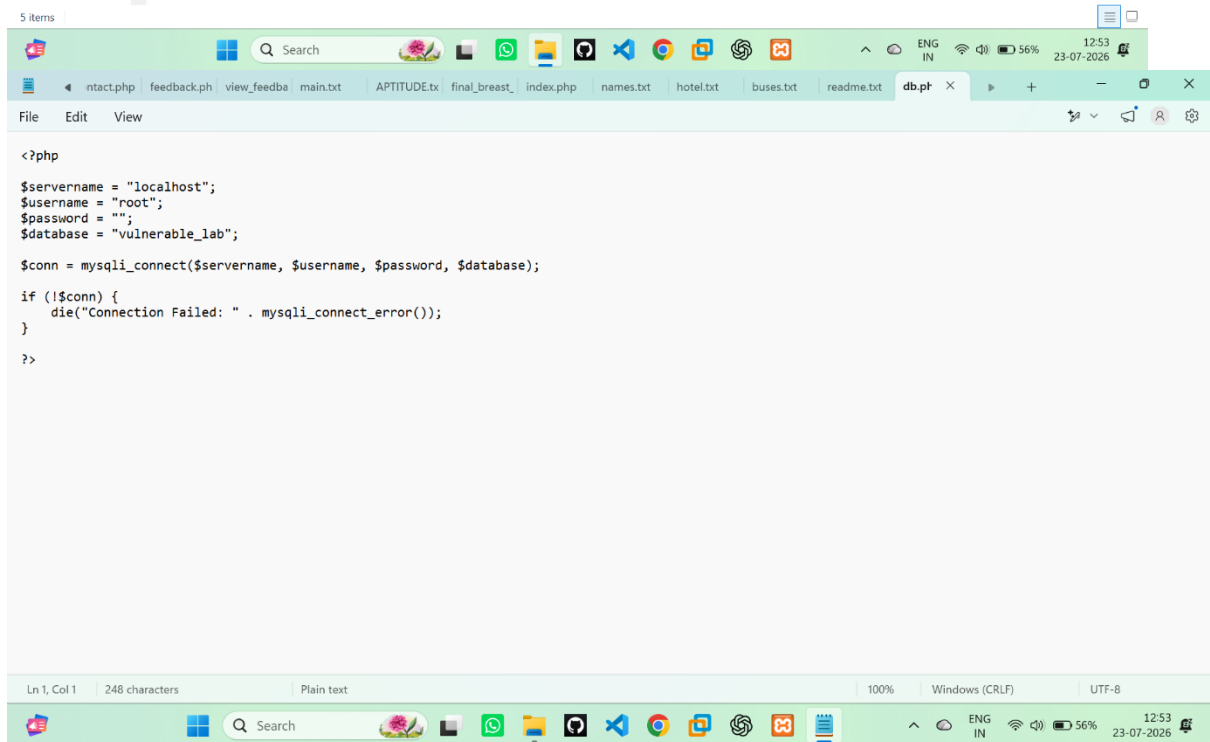
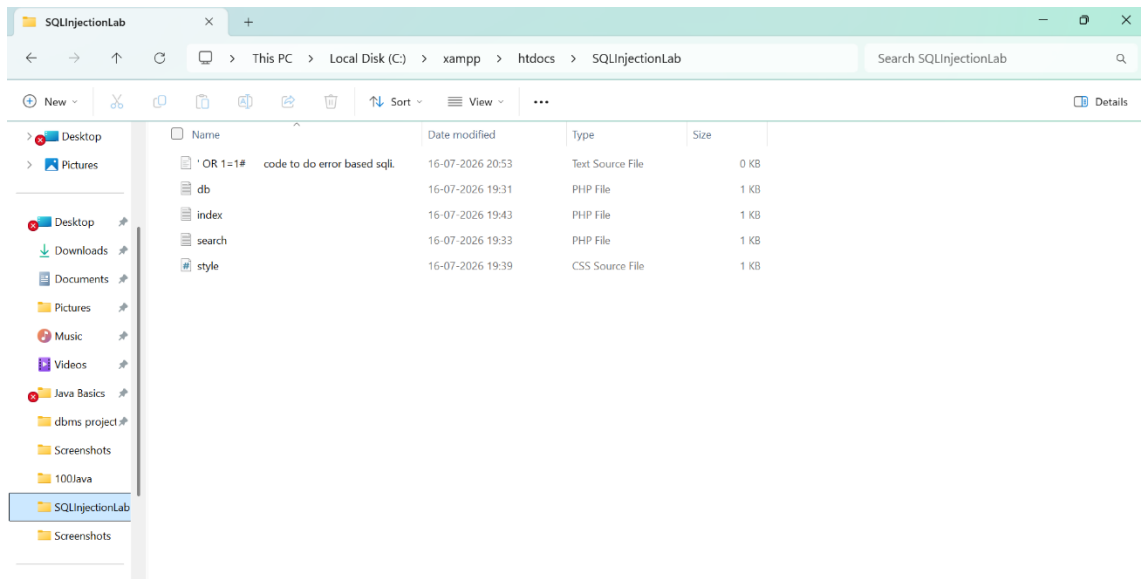
Conclusion

The **SQL Injection Lab** demonstrates how insecure handling of user input can introduce vulnerabilities into web applications.

By building an intentionally vulnerable search application using **PHP** and **MySQL** in a local sandbox, this project provides practical insight into how SQL Injection arises and why developers should avoid directly concatenating user input into SQL queries.

The project emphasizes secure software development awareness and serves as an educational exercise for understanding common web application security issues.

Project Demonstration



```
ntact.php | feedback.ph | view_feedba | main.txt | APTITUDE.tx | final_breast | index | names.txt | hotel.txt | buses.txt | readme.txt | db.php | + | - | X
File Edit View
<!DOCTYPE html>
<html>

<head>
<title>SQL Injection Lab</title>
<link rel="stylesheet" type="text/css" href="style.css">
</head>

<body>
<div class="container">

<h1>SQL Injection Lab</h1>

<form action="search.php" method="GET">

<input type="text" name="username" placeholder="Enter Username">
<h1>SQL Injection Lab</h1>

<p>
Educational Vulnerability Sandbox
</p>

<button type="submit">Search</button>

</form>
</div>

</body>

</html>

Ln 1, Col 1 | 492 characters | Plain text | 100% | Windows (CRLF) | UTF-8
12:53 23-07-2026
edback.ph | view_feedba | main.txt | APTITUDE.tx | final_breast | index.php | names.txt | hotel.txt | buses.txt | readme.txt | db.php | search | + | - | X
File Edit View
<?php
include("db.php");

$username = $_GET['username'];

// Search query
$sql = "SELECT * FROM users WHERE username='$username'";

$result = mysqli_query($conn, $sql);

echo "<h2>Search Results</h2>";

if (mysqli_num_rows($result) > 0) {

echo "<table border='1' cellpadding='10'>";
echo "<tr>
<th>ID</th>
<th>Username</th>
<th>Email</th>
<th>Password</th>
</tr>";

while ($row = mysqli_fetch_assoc($result)) {

echo "<tr>";

echo "<td>".$row['id']. "</td>";
echo "<td>".$row['username']. "</td>";
echo "<td>".$row['email']. "</td>";
echo "<td>".$row['password']. "</td>";

echo "</tr>";

}

echo "</table>";

} else {

Ln 1, Col 1 | 776 characters | Plain text | 80% | Windows (CRLF) | UTF-8
12:53 23-07-2026
```

File Edit Selection View Go ...

Search

style.css

C:\xampp\htdocs> SQLInjectionLab > style.css > body

```
1 body{
2   font-family: Arial, sans-serif;
3   background: #f4f4f4;
4   text-align:center;
5 }
6
7 .container{
8   width:600px;
9   margin:50px auto;
10  background: white;
11  padding:30px;
12  border-radius:10px;
13  box-shadow:0px 0px 10px gray;
14 }
15
16 input{
17   width:250px;
18   padding:10px;
19 }
20
21 button{
22   padding:10px 20px;
23   cursor:pointer;
24 }
25
26 table{
27   margin:auto;
28   border-collapse:collapse;
29   margin-top:20px;
30 }
31
32 th,td{
33   padding:10px;
34 }
```

Gemini Code Assist has been updated.
Source: Gemini Code Assist
View release notes

XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

XAMPP Control Panel v3.3.0

Config

Netstat

Shell

Explorer

Services

Help

Quit

Service	Module	PID(s)	Port(s)	Actions
☐	Apache	9692 2560	82303, 82304	Stop Admin Config Logs
☐	MySQL	11396	3306	Stop Admin Config Logs
☐	FileZilla			Start Admin Config Logs
☐	Mercury			Start Admin Config Logs
☐	Tomcat			Start Admin Config Logs

12:54:36 [main] You are not running with administrator rights! This will work for most application stuff but whenever you do something with services there will be a security dialogue or things will break! So think about running this application with administrator rights!

12:54:36 [main] XAMPP Installation Directory: "c:\xampp\"

12:54:36 [main] Checking for prerequisites

12:54:39 [main] All prerequisites found

12:54:39 [main] Initializing Modules

12:54:39 [main] Starting Check-Timer

12:54:39 [main] Control Panel Ready

12:54:43 [Apache] Attempting to start Apache app...

12:54:43 [Apache] Status change detected: running

12:54:53 [mysql] Attempting to start MySQL app...

12:54:53 [mysql] Status change detected: running

12:54:58 [mysql] Status change detected: stopped

12:54:58 [mysql] Error: MySQL shutdown unexpectedly.

12:54:58 [mysql] This may be due to a blocked port, missing dependencies, improper privileges, a crash, or a shutdown by another method.

12:54:59 [mysql] Press the Logs button to view error logs and check the Windows Event Viewer for more clues

12:54:59 [mysql] If you need more help, copy and post this entire log window on the forums

12:55:13 [mysql] Attempting to start MySQL app...

12:55:13 [mysql] Status change detected: running

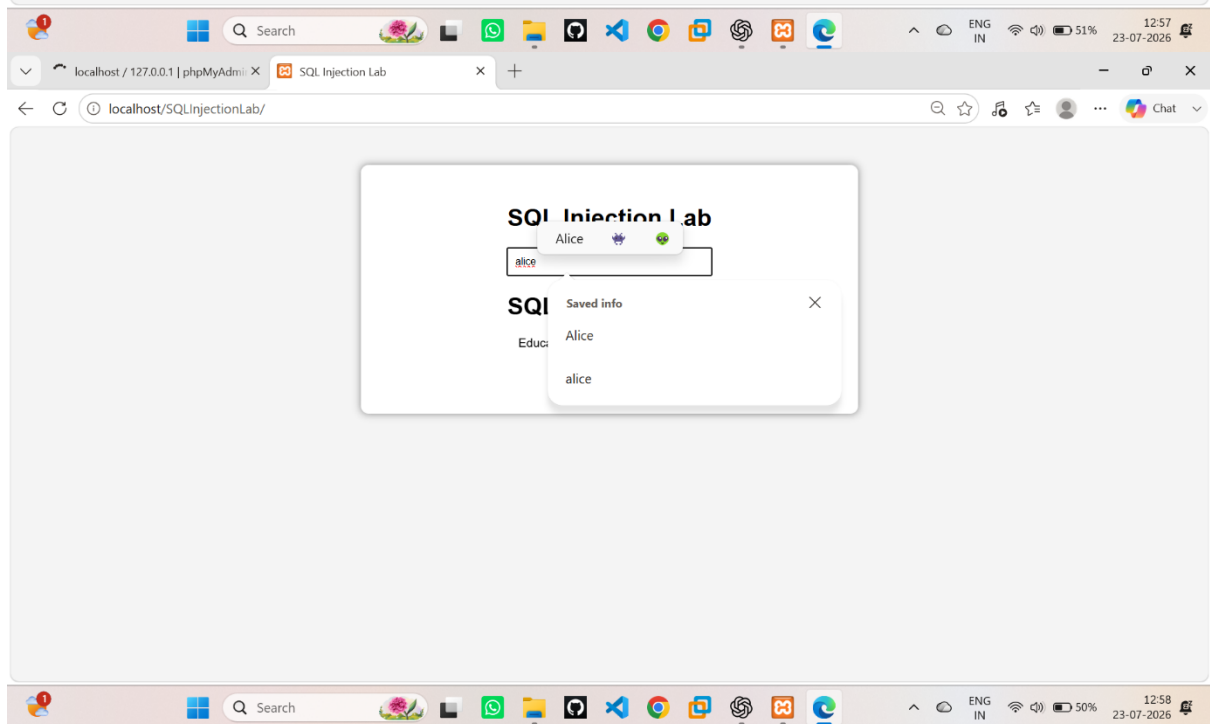
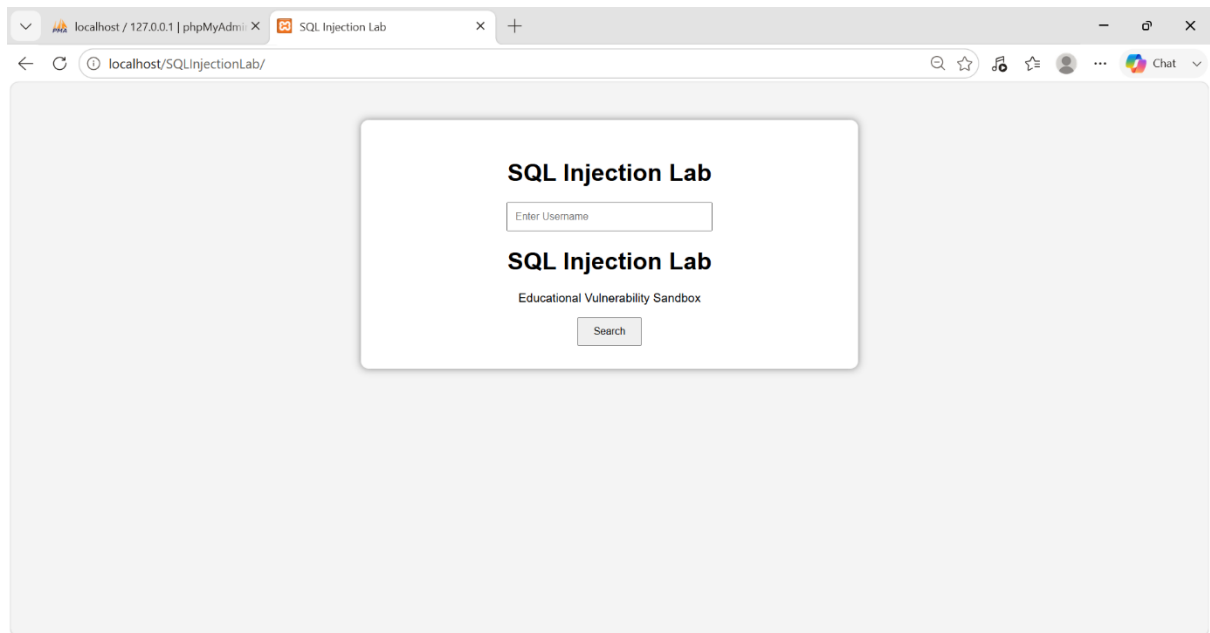
Search

ENG IN

55%

12:53

23-07-2026



localhost / 127.0.0.1 | phpMyAdmin

localhost/SQLInjectionLab/search

localhost/SQLInjectionLab/search.php?username=alice

Search Results

ID	Username	Email	Password
1	Alice	alice@gmail.com	alice123

localhost / 127.0.0.1 | phpMyAdmin

SQL Injection Lab

localhost/SQLInjectionLab/

SQL Injection Lab

SQL Injection Lab

Educational Vulnerability Sandbox

Search

localhost / 127.0.0.1 | phpMyAdmin X localhost/SQLInjectionLab/search X

localhost/SQLInjectionLab/search.php?username=%27+OR+1%3D1%23+

Search Results

ID	Username	Email	Password
1	Alice	alice@gmail.com	alice123
2	Bob	bob@gmail.com	bob123
3	Charlie	charlie@gmail.com	charlie123
4	David	david@gmail.com	david123
5	Emma	emma@gmail.com	emma123

localhost/SQLInjectionLab/search X Sri Anantashayana Borewells & Pu X localhost / 127.0.0.1 / vulnerable X

localhost/phpmyadmin/index.php?route=/sql&db=vulnerable_lab&table=users&pos=0

Server: 127.0.0.1 Database: vulnerable_lab Table: users

Showing rows 0 - 4 (5 total. Query took 0.0004 seconds)

```
SELECT * FROM `users`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

☐	Edit	Copy	Delete	1	Alice	alice@gmail.com	alice123		
☐	Edit	Copy	Delete	2	Bob	bob@gmail.com	bob123		
☐	Edit	Copy	Delete	3	Charlie	charlie@gmail.com	charlie123		
☐	Edit	Copy	Delete	4	David	david@gmail.com	david123		
☐	Edit	Copy	Delete	5	Emma	emma@gmail.com	emma123		

Check all With selected Edit Copy Delete Export

Show all Number of rows: 25 Filter rows: Search this table Sort by key: None

Query results operations

Print Console toolbar Export Display chart Create view

Console

Press Ctrl+Enter to execute query

```
>SELECT * FROM `users`
```